

Anno Accademico 2025/2026

Leonardo Donati

Ingegneria del Software
Reverse Proxy HTTP

Tesina.



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Sommario

Questo elaborato descrive la progettazione di un reverse proxy HTTP realizzato in Java, in grado di leggere le regole di instradamento da un *file di configurazione esterno* e di inoltrare ogni richiesta al servizio corretto sulla base del percorso richiesto. Oltre alla funzionalità di base di instradamento, il sistema è stato progettato per supportare più istanze dello stesso servizio a una singola rotta, introducendo la necessità di un meccanismo di *bilanciamento del carico*.

Indice

1	Introduzione	3
2	Analisi dei Requisiti	3
2.1	Requisiti Funzionali	3
2.2	Requisiti Non Funzionali	3
3	Stile Architetturale	4
3.1	Perché non uno Stile Event-Driven	4
3.2	I Livelli del Sistema	4
3.3	Linguaggio	4
3.4	Configurazione Dichiarativa delle Route	4
4	Modellazione UML	5
4.1	Use-Case Diagram	5
4.2	Class Diagram	5
4.3	Sequence Diagram	7
4.4	Activity Diagram	7
4.5	State Diagram	8
5	Design Pattern	10
5.1	Strategy Pattern	10
5.2	Singleton Pattern	11
5.3	Factory Method Pattern	11
5.4	Rispetto dei Principi SOLID	12
6	Conclusioni e Sviluppi Futuri	13
6.1	Sintesi dei Risultati	13
6.2	Sviluppi Futuri	13

1 Introduzione

Nei moderni sistemi web raramente un singolo servizio è responsabile della gestione di tutte le richieste in arrivo: applicazioni frontend, API REST, servizi di autenticazione e risorse statiche sono spesso ospitati su processi distinti. Un reverse proxy si colloca all'ingresso di questa infrastruttura, intercettando le richieste dei client e instradandole verso il servizio backend corretto, nascondendo al client la reale topologia del sistema.

2 Analisi dei Requisiti

2.1 Requisiti Funzionali

- **Routing delle Richieste HTTP:** il sistema deve ricevere una richiesta HTTP dal client e instradarla verso il servizio backend corretto, sulla base del percorso richiesto e delle regole definite in un file di configurazione esterno.
- **Load Balancing:** quando una rotta è associata a più istanze dello stesso servizio, il sistema deve distribuire le richieste tra le istanze disponibili secondo una strategia configurabile (ad esempio round-robin, selezione casuale, ecc), senza dover modificare il codice sorgente per cambiarla.
- **Risoluzione tramite Referer:** per le richieste di risorse statiche (stili css, script, immagini) prive di una ruota esplicita, deve essere necessario risalire al servizio di destinazione analizzando l'header **Referer** della richiesta, così da individuare la pagina che le ha originate.
- **Configurazione Dichiarativa delle Route:** deve essere possibile definire, tramite un file YAML esterno, l'insieme delle route disponibili e i servizi backend ad esse associati, senza dover ricompilare l'applicazione.
- **Gestione di Connessioni Concorrenti:** il sistema deve poter gestire più richieste client in maniera concorrente, ciascuna elaborata su un thread dedicato prelevato da un pool.

2.2 Requisiti Non Funzionali

- **Basso Overhead:** il tempo impiegato dal proxy per instradare una richiesta deve essere trascurabile rispetto al tempo di risposta del servizio backend.
- **Estendibilità delle Strategie di Bilanciamento:** deve essere possibile introdurre nuovi algoritmi di balancing del carico con il minor impatto possibile sulle classi esistenti.
- **Scalabilità della Configurazione:** l'aggiunta di nuovi servizi, istanze o route deve richiedere la sola modifica del file di configurazione.
- **Robustezza:** un errore nella comunicazione con un singolo backend o l'assenza di una route corrispondente, non deve compromettere la disponibilità del proxy per le altre richieste in corso.
- **Portabilità:** il software deve poter essere eseguito su qualsiasi piattaforma con una Java Virtual Machine, senza dipendenze dal sistema operativo sottostante.

3 Stile Architetture

Il reverse proxy è distribuito come un singolo processo Java, eseguito come un'unica unità deployabile. Non è un sistema distribuito, perché la logica applicativa, dall'accettazione delle connessioni fino all'inoltro della risposta, risiede all'interno della JVM. Per questo motivo, lo stile architetture adottato è quello del **Monolite a Livelli**, in cui le responsabilità del sistema sono suddivise in livelli logici ben distinti, ciascuno dei quali dipende solo dai livelli sottostanti attraverso interfacce astratte.

3.1 Perché non uno Stile Event-Driven

I reverse proxy utilizzati a livello enterprise sono tipicamente realizzati con un'architettura **Event-Driven**, basata su eventi e I/O non bloccante, capace di gestire un numero molto elevato di connessioni concorrenti. Ho comunque scelto di adottare il Monolite a Livelli per questo progetto, perché questa struttura è più facile da progettare, testare e descrivere rispetto a una asincrona a eventi. Inoltre, con il numero di richieste concorrenti di questo prototipo, un thread per connessione è la soluzione migliore, perché i benefici di scalabilità di uno stile event-driven diventano determinanti solo con carichi molto più elevati. L'utilizzo di un modello event-driven, con I/O non bloccante, può comunque essere un possibile sviluppo futuro del progetto.

3.2 I Livelli del Sistema

- **Livello di Connessione:** accetta le connessioni TCP in ingresso tramite il **ServerSocket** principale e delega la gestione di ciascuna connessione a un thread worker prelevato da un pool.
- **Livello di Parsing:** responsabile dell'analisi dello stream raw di byte proveniente dal socket del client, traducendo l'header HTTP in un oggetto che contiene metodo, URL e altri header in maniera strutturata.
- **Livello di Logica di Routing e Balancing:** incapsula la logica applicativa principale. In base alla route del percorso, seleziona il servizio nel backend corretto, usando la strategia di balancing dichiarata.
- **Livello di Configurazione:** gestisce le regole di instradamento caricate dal file YAML esterno, fornendo un'interfaccia agli altri livelli del sistema.

3.3 Linguaggio

Il linguaggio utilizzato è Java. La scelta è motivata dalla disponibilità di API per la gestione di socket e connessioni TCP, da un modello di concorrenza basato su thread semplice da utilizzare tramite un **ExecutorService** e dall'esistenza di librerie per il parsing dei file di configurazione, come SnakeYAML per i file YAML.

Trattandosi di un programma che introduce un ulteriore passaggio tra client e servizio, l'overhead introdotto dalla JVM rispetto a linguaggi più performanti come C/C++ risulta pienamente accettabile.

3.4 Configurazione Dichiarativa delle Route

La scelta di inserire le regole di instradamento in un file YAML esterno, caricato all'avvio dell'applicazione, evita che l'aggiunta o la rimozione di un servizio richieda una modifica del codice sorgente.

YAML Esempio di File di Configurazione

```
strategy: round-robin
```

```

routes:
  - path: /
    service:
      - host: localhost
        port: 8081
      - host: localhost
        port: 8082
  - path: /api
    service:
      - host: localhost
        port: 9090

```

Per ogni route, viene specificato il percorso, un host e una porta. Possono esserci più servizi associati a una route, per questo, il campo **strategy**, permette di scegliere l'algoritmo di load balancing da applicare senza intervenire sul codice.

4 Modellazione UML

Principali diagrammi che descrivono struttura e il comportamento dinamico del software.

4.1 Use-Case Diagram

- **Client / Browser:** l'entità che invia una richiesta HTTP al proxy.
- **Amministratore:** definisce le route nel file di configurazione.
- **Servizio Backend:** il sistema esterno che riceve la richiesta inoltrata ed elabora la risposta.

Tre cose da considerare:

- **Routing e Balancing:** il caso d'uso più importante, che include la ricerca della route corrispondente e la selezione della route con la strategia scelta.
- **Risoluzione tramite Referer:** caso d'uso che quando il percorso richiesto non corrisponde a nessuna rotta esplicita (tipico delle risorse statiche referenziate da una pagina già instradata).
- **Configurazione delle route:** l'amministratore definisce, tramite file YAML, l'insieme delle route e dei servizi disponibili.

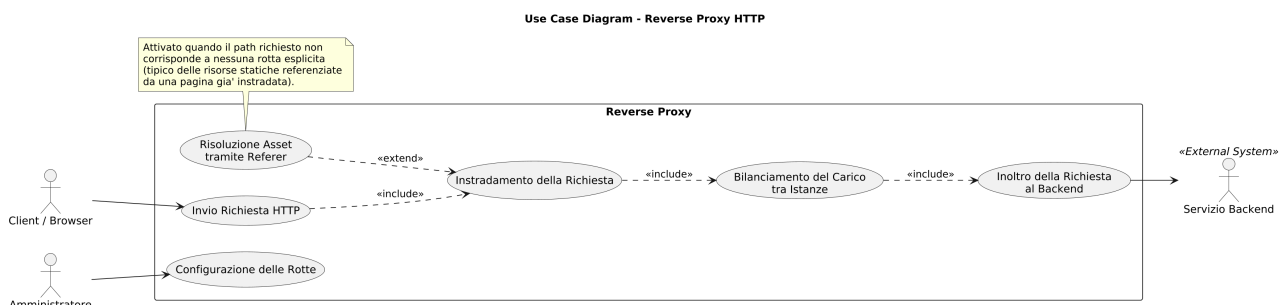


Figura 1: Use Case Diagram

4.2 Class Diagram

In questa sezione viene presentato il modello strutturale del sistema. Le classi principali sono:

- **ReverseProxy:** avvia il server e, per ogni connessione accettata, crea un nuovo **Client** a un pool di thread.
- **Client:** gestisce l'intero ciclo di vita di una singola connessione, dal parsing della richiesta fino alla chiusura dei socket.
- **HttpRequest:** rappresenta in maniera strutturata una richiesta HTTP, contenendo metodo, URL e header.
- **ConfigurationManager:** punto di accesso unico e globale alla configurazione caricata da file e alla strategia di balancing attiva.
- **Config, Route, Service:** il modello dati della configurazione creato direttamente dal file YAML. Una configurazione contiene più route, ciascuna delle quali può essere associata a uno o più servizi.
- **LoadBalancingStrategy e implementazioni:** l'interfaccia e le strategie concrete di selezione del servizio.
- **LoadBalancingStrategyFactory:** logica di creazione della strategia corretta.

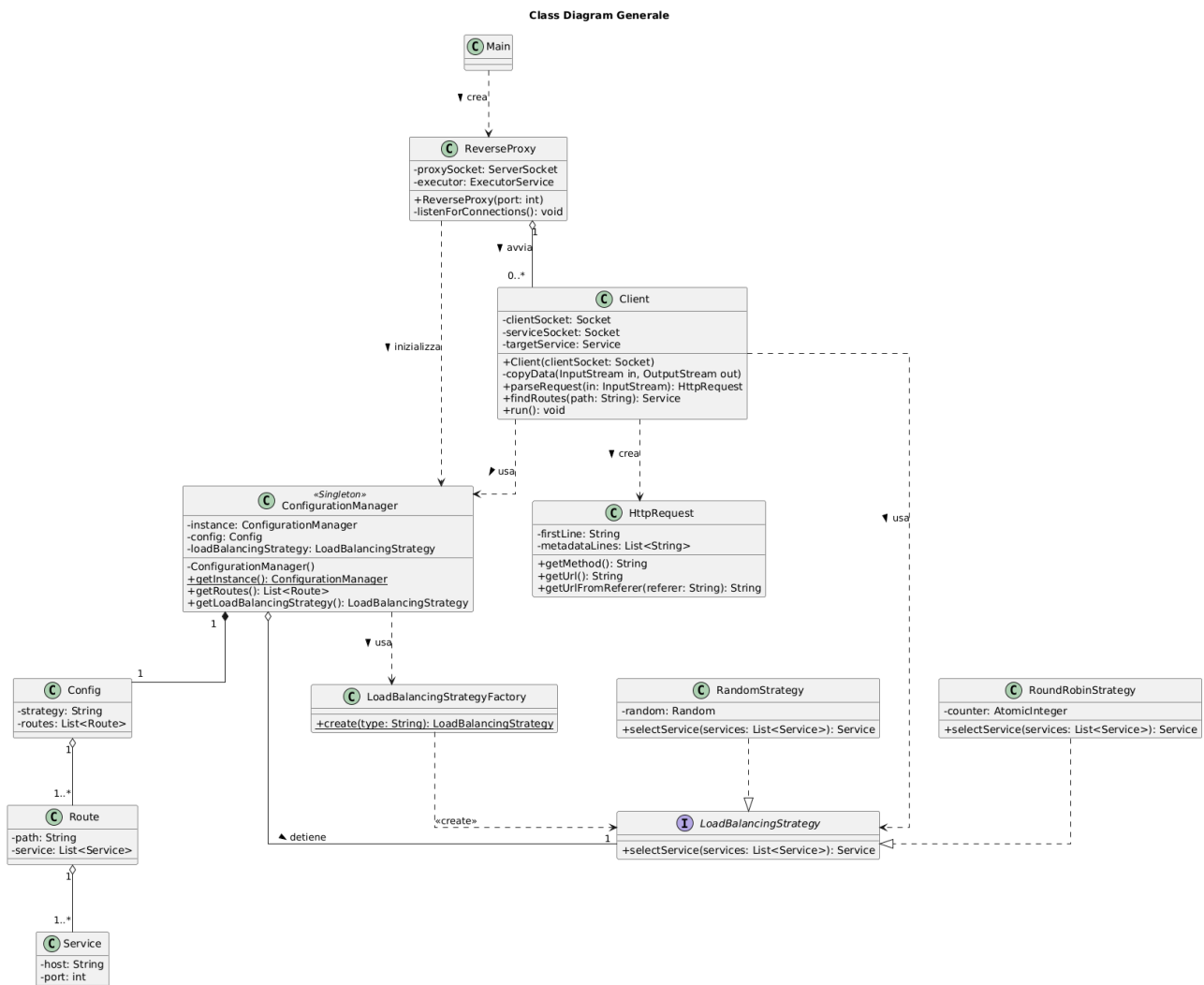


Figura 2: Class Diagram Generale

4.3 Sequence Diagram

Questo diagramma descrive lo scenario di inoltro di una richiesta verso una route con più istanze:

1. Il client apre una connessione TCP e invia la richiesta HTTP.
2. Il thread `Client` esegue il parsing della richiesta, ottenendo un oggetto `HttpRequest`.
3. Vengono recuperate le route configurate tramite `ConfigurationManager`.
4. Viene individuata la route corrispondente al percorso richiesto.
5. Se la route esiste, viene presa la strategia di balacing e utilizzata per selezionare il servizio.
6. Il thread `Client` apre una nuova connessione verso il servizio selezionato, inoltra la richiesta e poi la risposta ricevuta al client originario.

La selezione dell'istanza di servizio è completamente delegata all'implementazione della strategia, mantenendo il `Client` indipendente dall'algoritmo utilizzato.

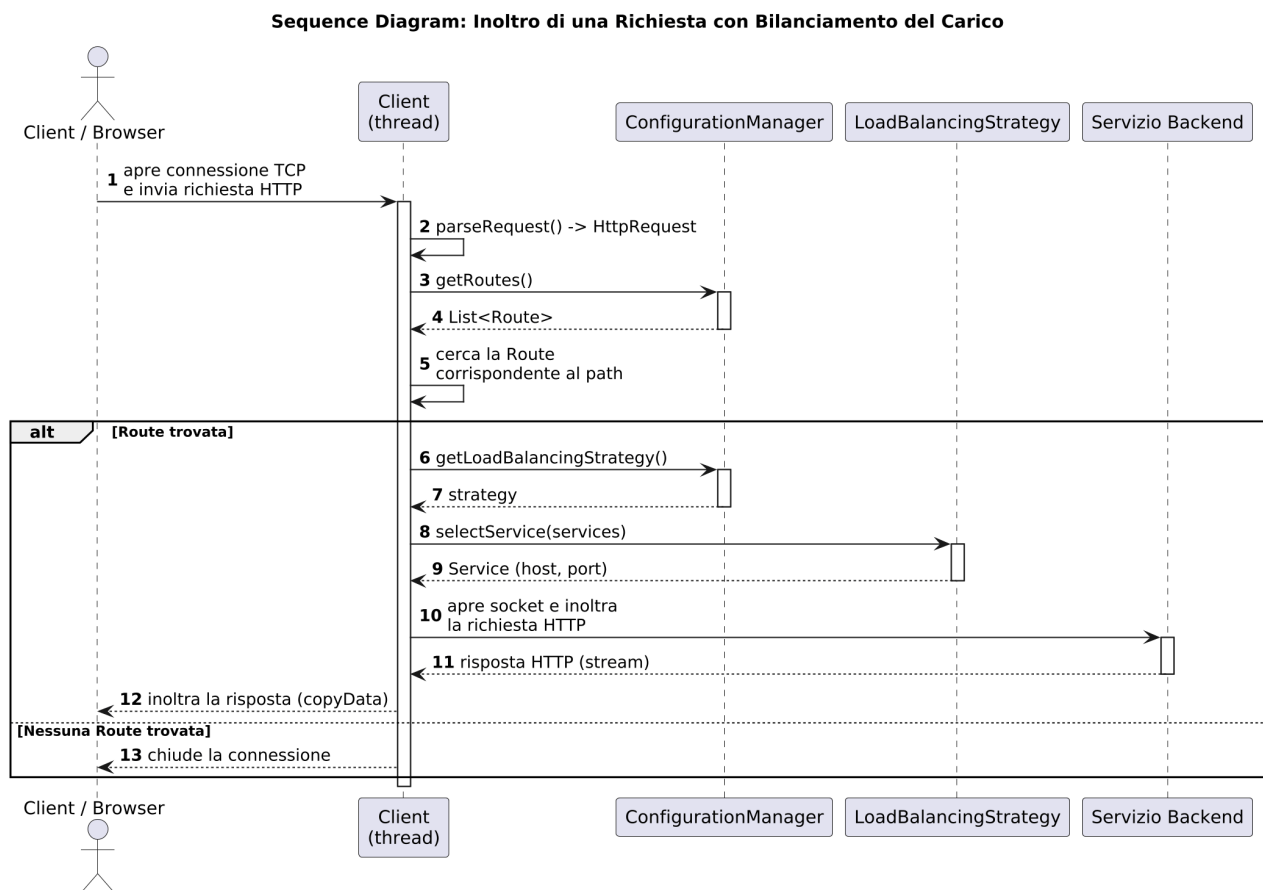


Figura 3: Sequence Diagram

4.4 Activity Diagram

Stesso scenario del diagramma precedente, ma rappresentato con un Activity Diagram.

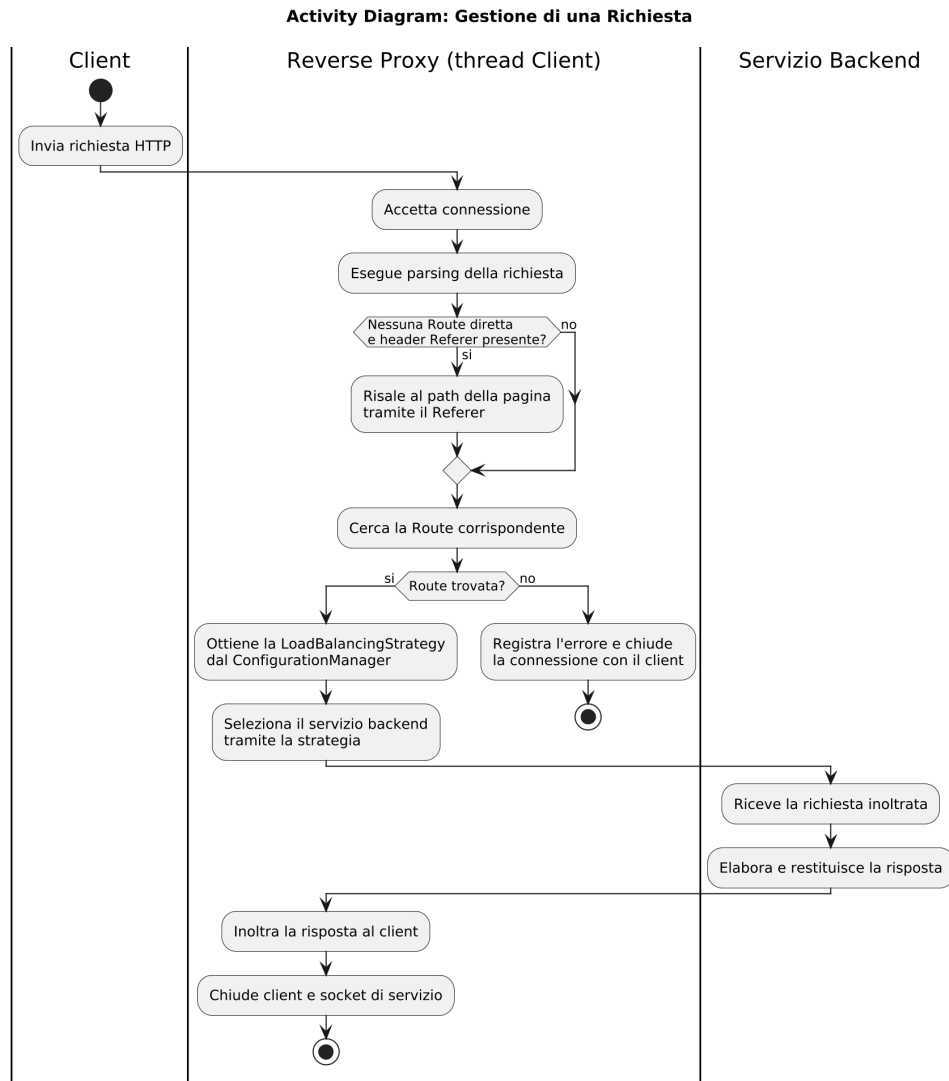


Figura 4: Activity Diagram

4.5 State Diagram

Il diagramma descrive tutti gli stati di una connessione gestita da un thread `Client`.

Gli stati principali sono:

- **Accettata:** stato iniziale, subito dopo l'accettazione della connessione da parte del `ServerSocket`.
- **InParsing:** la richiesta HTTP viene letta e trasformata in un oggetto `HttpRequest`.
- **InInstradamento:** viene ricercata la rotta corrispondente al percorso richiesto.
- **ConnessaAlBackend / InInoltro:** la connessione verso il servizio selezionato è stata aperta e la richiesta, poi la risposta, vengono inoltrate.
- **Fallita:** raggiunto in seguito a un errore di I/O o all'assenza di una route corrispondente.
- **Chiusa:** stato finale, raggiunto in ogni caso dal blocco `finally`, che garantisce il rilascio dei socket aperti indipendentemente dal risultato della richiesta.

State Diagram: Ciclo di Vita di una Connessione

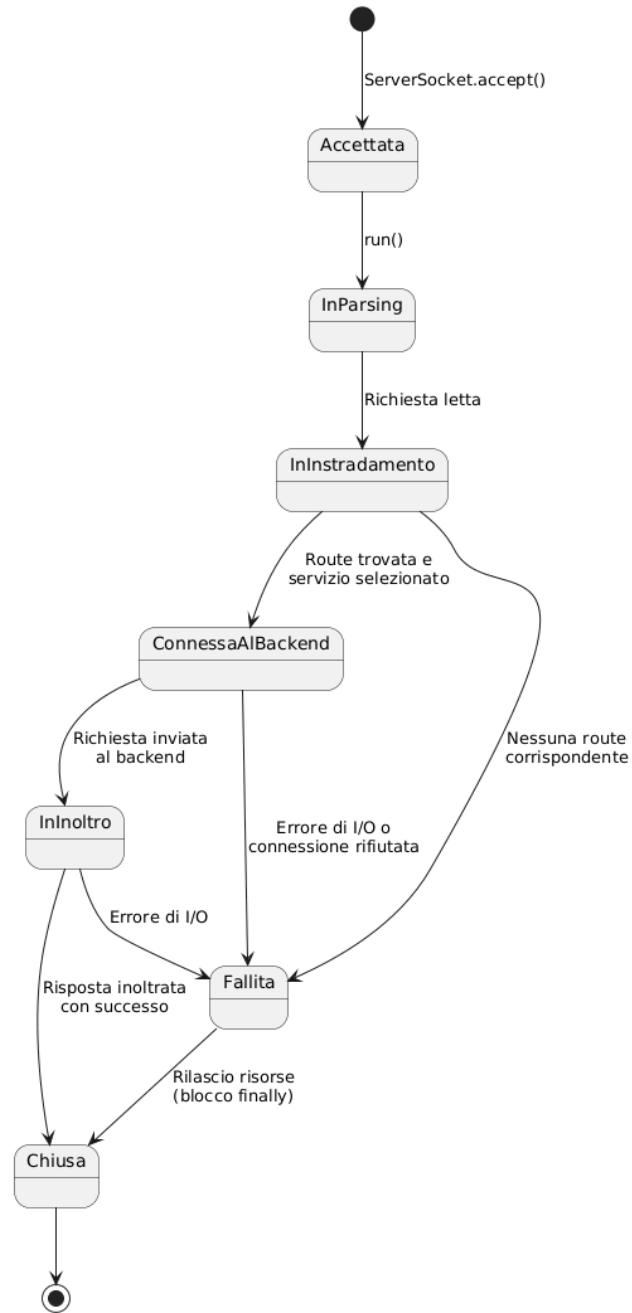


Figura 5: State Diagram per una Connessione

5 Design Pattern

In questa sezione vengono analizzati i design pattern utilizzati.

5.1 Strategy Pattern

Quando una route è associata a più istanze dello stesso servizio, il sistema deve decidere quale usare per gestire ciascuna richiesta. Dato che non esiste un unico algoritmo ottimo per ogni scenario, è stato adottato il pattern **Strategy**.

Questo pattern permette di definire una famiglia di algoritmi, incapsularli in classi separate, implementando un'unica interfaccia **LoadBalancingStrategy**. Questo li rende intercambiabili senza modificare il codice del **Client** che le utilizza.

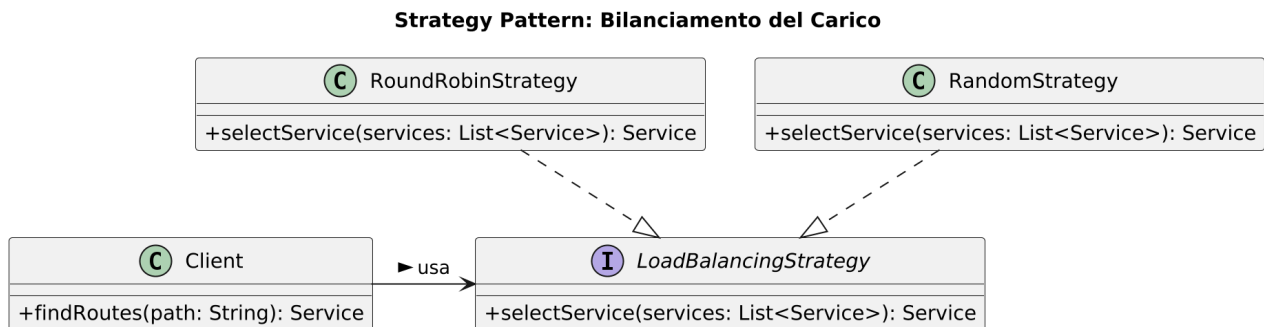


Figura 6: Class Diagram ridotto per mostrare lo Strategy Pattern

JAVA Implementazione Design Pattern

```
public interface LoadBalancingStrategy {
    Service selectService(List<Service> services);
}

public class RoundRobinStrategy implements LoadBalancingStrategy {
    private final AtomicInteger counter = new AtomicInteger(0);

    @Override
    public Service selectService(List<Service> services) {
        int index = counter.getAndIncrement() % services.size();
        return services.get(index);
    }
}

public class RandomStrategy implements LoadBalancingStrategy {
    private final Random random = new Random();

    @Override
    public Service selectService(List<Service> services) {
        return services.get(random.nextInt(services.size()));
    }
}

public class Client implements Runnable {
    public Service findRoutes(String path) {
        for (Route r : ConfigurationManager.getInstance().getRoutes()) {
            if (r.getPath().equals(path)) {
                LoadBalancingStrategy strategy =
                    ConfigurationManager.getInstance().getLoadBalancingStrategy();
            }
        }
    }
}
```

```

        return strategy.selectService(r.getService());
    }
}
return null;
}
}

```

5.2 Singleton Pattern

La configurazione delle route viene letta una sola volta all'avvio dell'applicazione, ma è condivisa da tutti i thread **Client** generati per servire le richieste contemporaneamente. Per evitare di ricaricare più volte il file YAML e per garantire che tutti i thread usino sempre la stessa configurazione, è stato applicato il pattern **Singleton** alla classe **ConfigurationManager**. Il costruttore privato impedisce la creazione di istanze al di fuori della classe stessa, mentre il metodo statico `getInstance()`, sincronizzato per essere sicuro anche in un contesto multi-thread, garantisce che la configurazione venga caricata al primo utilizzo e riutilizzata per tutta la durata dell'esecuzione.

JAVA Implementazione Design Pattern

```

public class ConfigurationManager {
    private static ConfigurationManager instance;

    private final Config config;
    private final LoadBalancingStrategy loadBalancingStrategy;

    private ConfigurationManager() {
        this.config = loadYamlConfig();
        this.loadBalancingStrategy =
            LoadBalancingStrategyFactory.create(config.getStrategy());
    }

    public static synchronized ConfigurationManager getInstance() {
        if (instance == null) {
            instance = new ConfigurationManager();
        }
        return instance;
    }

    public List<Route> getRoutes() {
        return config.getRoutes();
    }

    public LoadBalancingStrategy getLoadBalancingStrategy() {
        return loadBalancingStrategy;
    }
}

```

5.3 Factory Method Pattern

Il campo `strategy` del file di configurazione è una stringa, che deve essere tradotta nell'oggetto `LoadBalancingStrategy` concreto corrispondente. Per evitare che questa logica di creazione nel codice del `ConfigurationManager` o del `Client`, è stato usato il pattern **Factory Method**, incapsulato nella classe `LoadBalancingStrategyFactory`. Con questo pattern, per aggiungere un nuovo algoritmo è sufficiente creare una nuova implementazione di `LoadBalancingStrategy` ed estendere la factory con un nuovo caso, senza toccare altro.

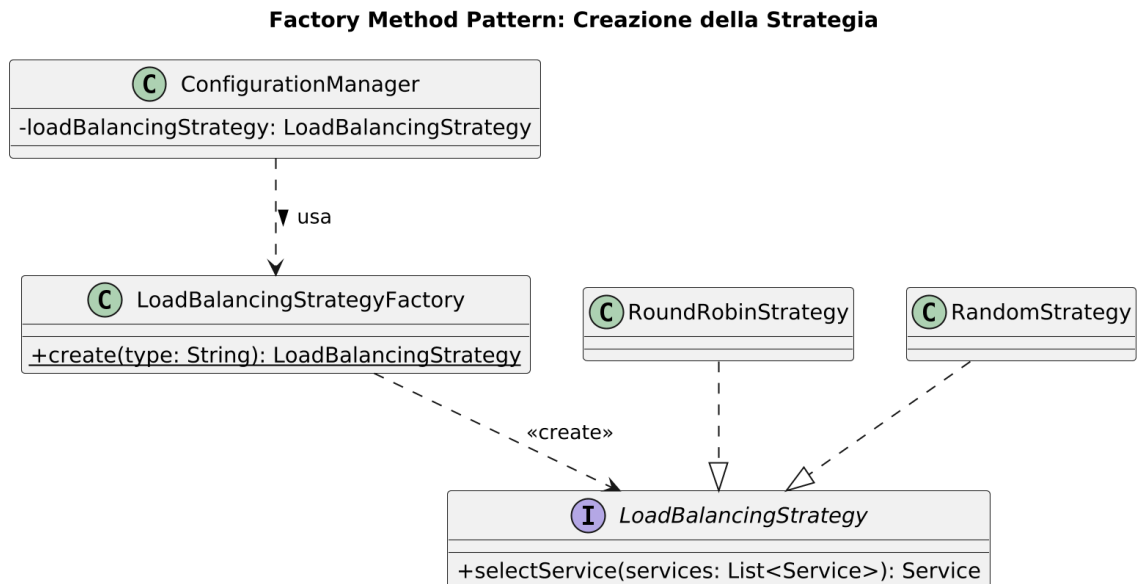


Figura 7: Class Diagram ridotto per mostrare il Factory Method Pattern

JAVA Implementazione Design Pattern

```

public class LoadBalancingStrategyFactory {
    public static LoadBalancingStrategy create(String type) {
        if (type == null) {
            return new RoundRobinStrategy();
        }
        switch (type.toLowerCase()) {
            case "random":
                return new RandomStrategy();
            case "round-robin":
            default:
                return new RoundRobinStrategy();
        }
    }
}
  
```

5.4 Rispetto dei Principi SOLID

La combinazione dei tre pattern descritti consente all'architettura di rispettare i principi SOLID:

- **Single Responsibility Principle:** ogni classe ha una singola responsabilità ben definita:
 - Client gestisce il ciclo di vita della connessione.
 - HttpRequest rappresenta la richiesta.
 - ConfigurationManager gestisce l'accesso alla configurazione.
 - Ciascuna LoadBalancingStrategy implementa un solo algoritmo di selezione.
- **Open/Closed Principle:** è possibile aggiungere nuove strategie di bilanciamento del carico creando nuove classi che implementano LoadBalancingStrategy, senza modificare altro.
- **Liskov Substitution Principle:** qualunque implementazione dell'interfaccia LoadBalancingStrategy può essere utilizzata al posto di un'altra senza alterare la correttezza di Client.

- **Interface Segregation Principle:** l'interfaccia `LoadBalancingStrategy` espone un solo metodo, evitando implementazioni inutili.
- **Dependency Inversion Principle:** `Client` dipende dall'astrazione `LoadBalancingStrategy` e non dalle sue implementazioni concrete.

6 Conclusioni e Sviluppi Futuri

6.1 Sintesi dei Risultati

Il lavoro svolto ha portato alla progettazione di un reverse proxy HTTP capace di instradare le richieste verso i servizi backend corretti sulla base di un file di configurazione dichiarativo.

- L'uso dello **Strategy Pattern** ha reso il sistema flessibile, permettendo di cambiare l'algoritmo di load balancing agendo esclusivamente sul file di configurazione.
- L'uso del **Singleton Pattern** ha garantito un punto di accesso unico alla configurazione condivisa tra i thread concorrenti.
- L'uso del **Factory Method Pattern** ha isolato la logica di creazione delle strategie di balancing, semplificando l'introduzione di nuovi algoritmi in futuro.

6.2 Sviluppi Futuri

Nonostante il sistema attuale rappresenti una base solida e volutamente semplice, sono possibili diverse estensioni:

1. **Utilizzo di I/O Non Bloccante:** sostituire il modello a thread-per-connessione basato su Java NIO, così da avvicinare il sistema allo stile Event-Driven usato dai reverse proxy enterprise.
2. **Health-check dei Servizi Backend:** introdurre un controllo periodico dello stato delle istanze configurate, così da escludere automaticamente dal pool utilizzato dalla `LoadBalancingStrategy` i servizi non raggiungibili.
3. **Connessioni Persistenti:** supportare le connessioni HTTP keep-alive, oggi non gestite poiché ogni connessione viene chiusa dopo l'inoltro di una singola richiesta.
4. **Funzionalità per Prestazioni Migliori:** l'introduzione di un ulteriore livello al proxy permetterebbe di aggiungere funzionalità come il caching delle risposte o il rate limiting, senza modificare la logica di inoltro esistente.
5. **Sicurezza:** aggiungere il supporto a connessioni TLS/HTTPS in ingresso.
6. **Monitoraggio:** introdurre un sistema di logging strutturato e metriche per facilitare il monitoraggio del sistema in un contesto di produzione.